Trajectory Planning for the UFactory Lite 6 Robot Arm via the Official Python SDK

Author: Miroslav Denkov

Table of Contents

---

1. Abstract

This paper explores a possible solution for trajectory planning for the UFactory Lite 6 Robot arm model. Using the Python SDK, a developer can programmatically set trajectory points using a coordinate system or manual trajectory recording. The author has made use of the first. Software logic flowchart and a piece of code is provided and can be downloaded here(Refer to Appendix for links). In the end a small demo is procured and is concluded that, the Python SDK is quite nice and simple to work with, though not without compromise.

2. Introduction

This document aims to shed light on the workflow and intricacies involved in writing software used to for trajectory planning and job execution for the *UFactory Robot arm*, model *Lite 6*. We will be looking at it from the context of the Big Chemistry project. In the following chapters, you will notice the different types of DOT research methods that come into play for the duration of the project's task.

3. What is UFactory Lite 6 Robot arm?

As the name suggests, the machine is a mechanized non-humanoid arm that is operated with manufacturer's own software or can be programmatically set to execute instructions using, again, a Standard Development Kit from UFactory(manufacturer). The program instructions are written in

Python and UFactory provides a web interface that connect's to the arm's server, which includes a Python IDE, and allows a developer to write logic for the robot, much like how one would write software for an Arduino, using their API.

4. Why the Lite 6?

The job of the arm is to transport low-weight payload(a small spot plate) from an Opentrons Pippeting Robot, to a Tecan Spark plate reader. The scientists that will use this setup for their daily lab activities must have reliable equipment that allows them to handle chemical substances with great care. The Lite 6 model is one of the best budget options UFactory provides and its low-weight payload handling(up to 600 grams) is a great match for what the robot's function will be.All with having a cost of 4200$ for an entire set(gripper, vacuum gripper etc.),quite cheap compared to other models.

As mentioned above, UFactory also provides users with a Python SDK that allows engineers to control and automate arm functions with code. The software is open-source and transparent and can be accessed by anyone from UFactory's website, leading to their Github page.

5. How the Python SDK can be leveraged for trajectory planning for UFactory Lite 6?

The SDK is meticulously crafted by the individuals at UFactory, for the sole purpose of easily writing software for the arm. Utilizing the extensive functionality of the API is a vital part of making the hardware do what one needs it to do.

The only way to explore what the SDK is capable of, is to practically write code and experiment.

5. 1. Documentation

UFactory, fortunately for most, has some very detailed documentation and thanks to its open-source nature; we can make use of the fact that we can read code and edit it as well(not that we need to, but it is good nontheless). In the Github page (click here), we can find the entirety of the API, directly accessed from UFactory's website. Two options are presented before us, one is to install the Python API directly and use it in our IDE of choice, supplying the IP of the machine we are to operate and we can programmatically use it. The other option is to use UFactory's own web application, UFactory Studio.

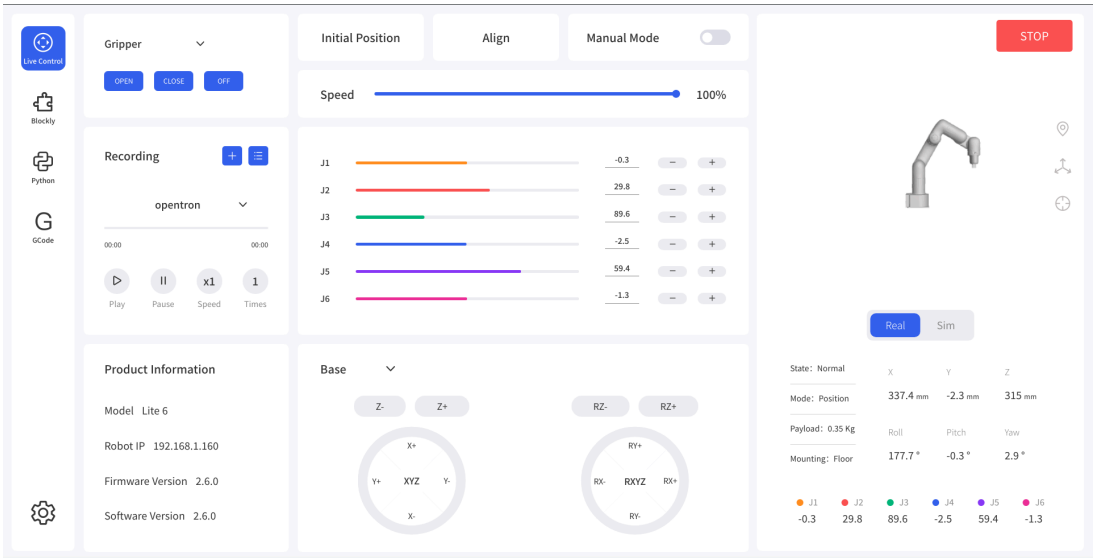

Figure 1: UFactory Studio interface

UFactory Studio is an application that has everything one might need to work with their robots. The home(live control) section contains all the tools one would need to manually control the robot arm. Additionally, each joint returns it's values in degrees and a 3D model of the product is present on the right side for visualization. An operator can choose whether to operate the real-hardware or instead just use the built-in simulation(though not as realistic as other solutions, such as Gazebo or NVidia's Isaac Sim; it proves as a very good low-resource simulation solution). We can move the arm, the model or both across a coordinate system on 3 axises(x,y,z), as shown below.

We will explore the other sections as we continue the research.

5. 2. Programming the UFactory Lite 6 using the official Python API

UFactory Studio also provides its own IDE directly in the application. It allows for three different ways to send instructions to the arm (UFactory's own Blockly script, Python and G-code). In this section we will be exploring only the Python interface, as it offers much more granular control, at the cost of increased complexity.

What makes things easier is that there are plenty of demonstrations that showcase the arm's capabilities. Using them, one can make out exactly how the Application Interface works.

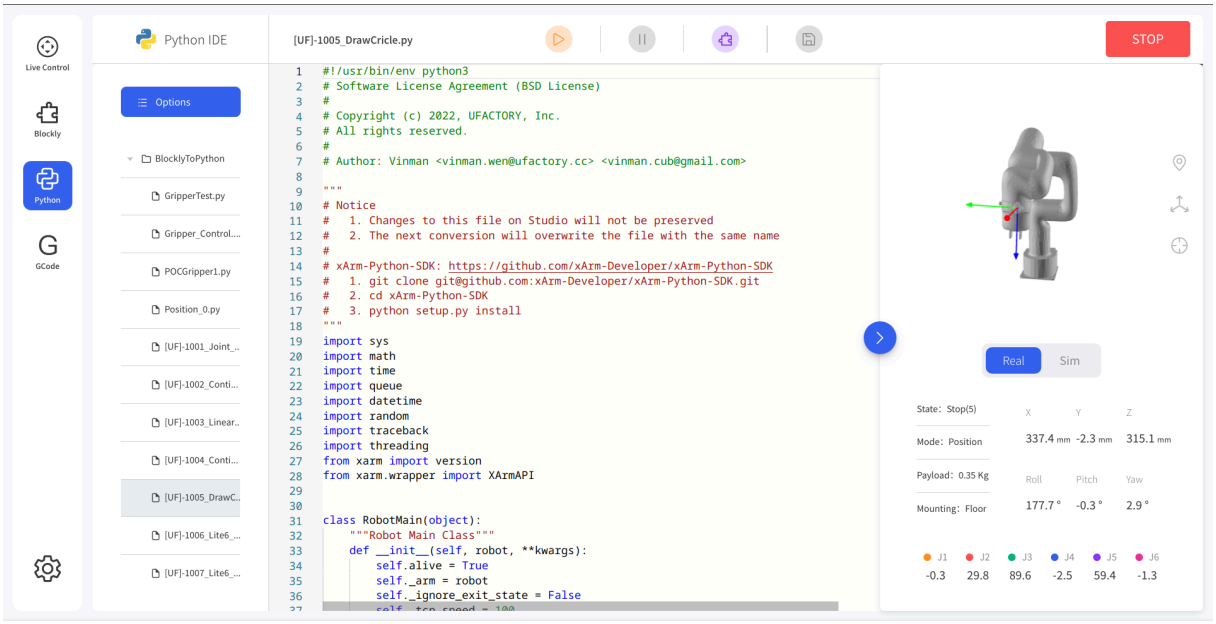

Figure 2: Circle drawing demo code.

The interface is very intuitive and anyone familiar with Python will feel quite at home. It is important to know, according to my findings, it is beneficial for one to familiarize themselves with embedded system's concepts, before programming the arm. Another thing of note is that the application provides a wait parameter for most functions, which is used to make timing easier, the parameter's job is to make sure the arm has completed its instruction before moving onto the next, but is not fully reliable so using one's own delay() function or equivalent thereof is recommended.

5. 3. How to program a pickup demo for UFactory's Lite 6 in Python

To learn and eventually make the arm perform the entire choreography, we must set up a prototype demo. The design is very simple and follows the following logic:

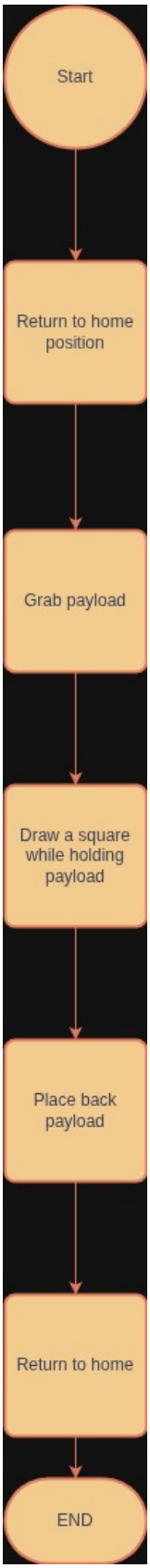

Figure 3: Demo logic

The code itself is written in UFactory studio, all in one Python file. Each piece of logic is written in its own separate function which are then combined in a job function that acts as the synchronizer between. The code should be very self-explanatory

```python
def job():
    reset_to_home() #reset robot to starting position VERY IMPORTANT!!!!!
    move_to_pickup()
    pick_up()
    draw_square()
    reset_to_home()
    arm.stop_lite6_gripper(sync=False)
    arm.disconnect()
```

Figure 4: Job function's logic in code

Using that logic we can make the robot safely and reliably pick up the payload and move it around, before setting it back in place. The physical setup is as follows:

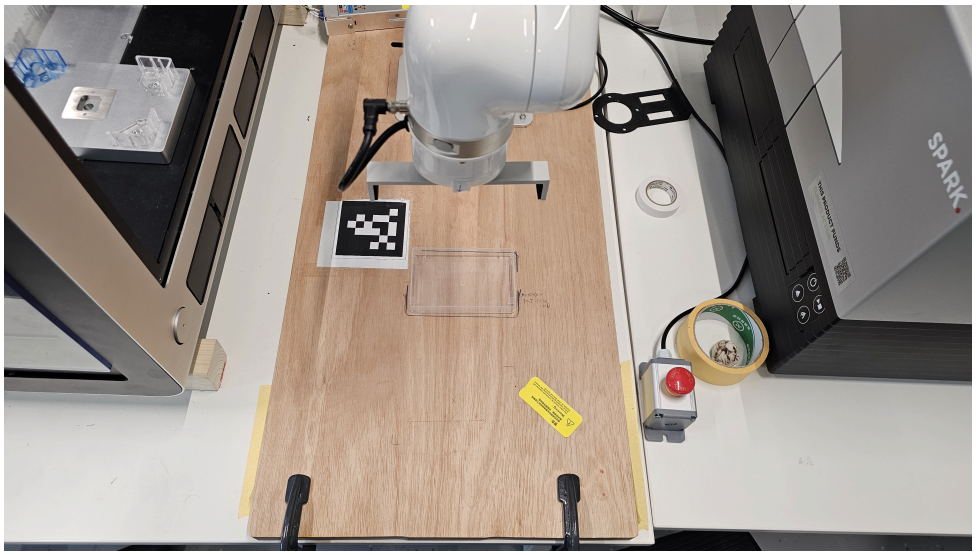

Figure 5: Physical setup

Programmers should be aware that the default "wait" delay is mal-functional and the arm executes instructions too fast, causing it to miss entire sections of logic, thus a manual delay using Python's **time** library is needed. A copy of the demo's code can be downloaded (Refer to Appendix).

5. 4. How does the trajectory planning work, using UFactory Studio?

Trajectories can be planned in two ways. One, to teach the arm using the special study mode. This mode allows one to record a trajectory and then replay it, though it is not tested by me(the author), yet. The other is to manually (in code) plan trajectory using small points(represented by coordinates) that the arm needs to go through to get to the location it needs. This is manual and as of yet, there is no solution to automate the entire process. One way to do it would be to use ROS, but this will be addressed in another paper.

---

6. Conclusion

To conclude, the Python API is very well implemented and makes writing code for the arm very simple, bypassing usual embedded issues such as setting up tick time, timers or CPU clocks. The high-level abstraction is very useful for simplicity, yet robs the user of functionalities that otherwise might be necessary, though using API libraries from Python would fix most issues; especially related to timing. Usually if you must use UFactory Studio, you would lose a lot of IDE features you may rely

on, it is recommended that UFactory Studio is used for its built-in simulator and then move the code to another IDE for extending capabilities, if needed.

7. Appendix

7. 1. DOT Framework symbols

In this document, you will see icons such as , without any follow-up text like Python. This is done to maximize information output with minimal lexical hindrance.

The following table explains all DOT research methods and combines them with their icon so that the reader may easily find and reference. The aim of the DOT symbols specifically is to show what research methods were involved without cluttering the text with unnecessary noise and to avoid repetition. The descriptions are taken from the official source (click here). Please study the table to learn about the symbols used herein:

DOT Method	Description	Symbol
Library	"Library research is done to explore what is already done and what guidelines and theories exist that could help you further your design. Since the advent of the internet library research is also called desk research."	
Field	"Field research is done to explore the application context. You apply a field strategy to get to know your end users, their needs, desires and limitations as organizational and physical contexts in which they will use your product."	
Lab	"Lab research is done to test your ideas with the users of your product. You use lab research to learn if things work out the way you intended them."	
Showroom	"Showroom research is done to test your ideas in relation to existing work. Showing your prototype to experts can be a form of showroom research or spelling out how your product is different from the competition."	
Workshop	"Workshop research is done to explore opportunities. Prototyping, sketching and co-creation activities are all ways to gain insights in what is possible and how things could work."	

7. 2. Download links

Source code.
Video demo.

7. 3. Bibliography

[1] J. Vogel, "ICT Research Methods — Methods Pack for Research in ICT." [Online]. Available: https://ictresearchmethods.nl/

[2] “XArm-Python-SDK.” [Online]. Available: https://github.com/xArm-Developer/xArm-Python-SDK

[1], [2]